

Volume = storage attached to a Pod.

But normal Pod storage is temporary.

When Pod dies, data can disappear.

So Kubernetes gives us:

PersistentVolume = actual storage available in cluster

PersistentVolumeClaim = application asking for storage

Pod = uses the claim and mounts it like a folder

Think like this:

```
Pod says:  
"I need 1GB storage"  
  
PVC says:  
"Give me 1GB storage"  
  
PV says:  
"Here is actual 1GB disk/path/storage"  
  
Container uses it at:  
/app/data
```

For our meal app, we can store meals in a file or SQLite DB at:

```
/app/data/meals.db
```

Without persistent volume: data gone when Pod restarts.

With persistent volume: data stays.

Next we should do this flow:

```
Step 1: Understand normal volume  
Step 2: Use emptyDir  
Step 3: See why it is temporary  
Step 4: Create PersistentVolume  
Step 5: Create PersistentVolumeClaim  
Step 6: Mount PVC into Deployment  
Step 7: Modify Go app to write/read data
```

We can start with **Volume vs Persistent Volume** first.

Yes

Good. First understand this properly because most Kubernetes confusion starts here.

1. Normal Container Storage

Suppose your Go app runs inside a Pod.

Inside container:

```
/app
```

You create a file:

```
/app/meals.txt
```

Looks fine.

But Pods are temporary.

If:

- Pod crashes
- Deployment recreates Pod
- Scaling happens
- Node dies

then that container filesystem is recreated.

Your file disappears.

That is called:

```
Ephemeral storage
```

Temporary storage.

2. Kubernetes Volume

Kubernetes introduced:

```
volumes:
```

A volume is storage attached to Pod.

Example:

```
volumes:
  - name: meal-storage
    emptyDir: {}
```

This creates storage shared inside Pod containers.

Then mounted:

```
volumeMounts:
  - name: meal-storage
    mountPath: /app/data
```

Now your app can write:

```
/app/data/meals.txt
```

3. What is emptyDir?

`emptyDir` is the simplest volume.

Kubernetes creates an empty directory when Pod starts.

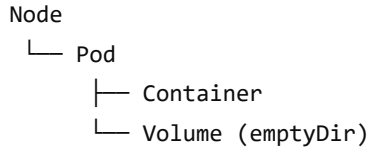
Important:

- Pod survives → data survives
- Container restarts → data survives
- Pod deleted → data gone

Very important distinction.

Container restart $\neq$ Pod restart.

4. Architecture



Volume belongs to Pod.

Not container.

That's why multiple containers inside same Pod can share files.

5. Real Use Cases of Normal Volumes

Shared logs

Container A writes logs
Container B reads/processes logs

Cache

Temporary cache storage.

Scratch space

Temporary processing files.

6. Why PersistentVolume Exists

Problem:

`emptyDir` dies with Pod.

So Kubernetes introduced cluster-level storage.

PersistentVolume (PV)

This survives Pod recreation.

Can be:

- AWS EBS
- Azure Disk
- NFS
- Local disk
- Ceph
- Azure Files

etc.

7. Real Flow

```
graph BT; DP[Deployment/Pod] --> PVC[PersistentVolumeClaim]; PVC --> PV[PersistentVolume]
```

PersistentVolume
↑
PersistentVolumeClaim
↑
Deployment/Pod

Pod never directly asks for storage.

It asks through PVC.

Exactly like cloud resource abstraction.

8. Very Important Mental Model

Volume

Pod-level temporary storage.

PersistentVolume

Cluster-level durable storage.

PVC

Request ticket for storage.

